

1) Which procedure executes faster, a compiler or pure interpreter? Explain.

Compiler is faster as pure interpreter reads the source code line by line and executes it as it goes while compiler takes the source code and converts it into machine language.

2) Define the meaning of syntax and semantics.

- **Syntax:** The structure of the expression, statement and program unit.
- **Semantics:** The meaning of the expression, statement and program unit.
- **Syntax and Semantics:** provide a language definition.

3) What is a variable? [list and explain all the attributes of a variable]

Variable is an abstraction of a computer memory cell or collection of memory cells

- **Name:** Is a string of characters used as an identifier.
- **Address:** Is the machine memory address with which it is associated with.
- **Type:** Is the range of values that a variable can store.
- **Value:** Is the content of the variable.
- **Lifetime:** Is the time duration in which the variable is bounded to a specific memory location.
- **Scope:** Defines where variables can be accessed or referenced.

4) What is the definition of block?

Is a compound statement that lets you group any number of data declarations into one statement using curly brackets {}.

5) What does it mean when a variable is static?

Static variables are those that are bounded to the memory cells before program executions begins and will remain bounded until the program execution terminates.

Advantage: Efficiency.

Disadvantage: Reduced flexibility.

6) what are the design issues for multiple-selection statements?

- What is the form and type of the expression that controls the section?
- How are the selectable segments specified?
- Is execution flow through the structure restricted to include just a single selectable segment?
- How are the case values specified?
- How should unrepresented selector expression values be handled, if at all?

7) How does C support Relation and Boolean expressions?

C supports Relation expressions by using signs such as >, >=, <, <=, ==.

And supports Boolean expressions by using 1 and 0 to represent true and false where 1 is true and 0 is false.

8) What are the two common problems with pointers?

- **Dangling pointer:** When an object with an incoming reference is deleted or deallocated without modifying the value of the pointer, leaving the pointer still pointing to the memory location of the deallocated memory.
- **Lost Heap-Dynamic Variable:** An allocated Heap-Dynamic Variable that is no longer accessible to the user program, but still contains some useful data
(Often called “garbage”).

9) In what way is C's “for” statement more flexible than that of other languages?

- All 3 expressions are optional.
- Each expression can be entire statements or sequence.
- Expression value is the value of the last statement.
- Everything can be changed within the loop.
- Legal to branch into the body of the loop.

10) Define the subprogram protocol.

Is the parameter profile plus if it's a function its return type.

11) Define the referencing environment.

Referencing environment: The collection of all the variables that are visible in the statement.

Static-scoped language: Is the local variable, plus all visible variables in all enclosing scopes.

Dynamic-scoped language: Is the local variable, plus all visible variables in all active subprograms.

12) what does it mean when a variable is Implicit Heap Dynamic?

Implicit Heap-Dynamic variable is bounded to heap storage when the variable is assigned a value.

Advantage: They have the highest level of flexibility, allowing highly generic codes to be written.

Disadvantage: The loss of some error detection by the compiler.

13) What does it mean when a variable is Explicit Heap Dynamic?

Are nameless memory cells that are allocated and deallocated by explicit run-time instructions written by the programmer.

Advantage: Providing Dynamic storage management.

Disadvantage: Inefficient and Unreliable.

14) what are the design issues for arrays?

- What type are legal for subscripts?
- When are subscript ranges bound?
- When does array allocation take place?
- Are ragged or triangular multi-dimensional arrays allowed?

- Can array be initialized when they have their storage allocated?
- What kind of slices are allowed?

15) What is unusual about C's multiple selection statement?

The C switch statement has virtually no restrictions on the placement of the case expressions, which are treated as if they were normal statement labels. This laxness can result in highly complex structure within the switch body. The reliability issue with the design arises when the mistaken absence of a break statement.

16) what are the three semantic models of the parameter passing?

- **In-mode:** Formal parameter can receive data from actual parameter.
- **Out-mode:** Formal parameter can transmit data to actual parameter.
- **In-Out-mode:** Can do both.

17) Define the subprogram signature?

The parameter profile of the subprogram contains the number, order and types of its formal parameter.

18) How does short-circuit expression evaluation work?

A short-circuit of an expression is one in which the result is determined without evaluating the whole expression. EXAMPLE $(a*b)*(12+b)$ $a = 0, b = 2$.

19) How does lexical analyzer a part of syntax analyzer?

A lexical analyzer server as a front-end to the syntax analyzer.

- **Lexical analyzer:** acts as an interface between the source code and the rest of the phases of a compiler.

- **Syntax analyzer:** Takes the input from the lexical analyzer in the form of token stream.

20) Define actual parameter and formal parameter.

- **Actual parameter:** Is the variables passed while calling a function.
- **Formal parameter:** There are the variables declared in the function, and receive their values when a call to the function is made.

21) What is stack dynamic variable?

Is one that is bounded to an address on the stack which is dynamically allocated during run-time.

22) What are the differences between lexical and syntax analysis?

- **Simplicity:** Techniques for lexical analysis are less complex than those required for syntax analysis.
- **Efficiency:** Syntax analyzer is more efficient as lexical analyzer requires a significant portion of total completion time
- Lexical analyzer is platform dependent; Syntax analyzer is platform independent
- Lexical analyzer is the first phase of compilation; Syntax analyzer is the second phase of compilation.

23) What is the generic subprogram and overloaded subprogram?

- **Generic subprogram:** Is one whose computation can be done on data of different types and on different calls.
- **Overloaded subprogram:** Is one that has the same name as another subprogram in the same referencing environment.

24) What is coroutine?

It's a computer program component that generalize subroutines for non-preemptive multitasking, by allowing execution to be suspended and resumed.

25) What is operator overload?

It's a potential problem in which a single operator symbol has more than one meaning.

26) What is reserved word?

A special word in a programming language that has a fixed meaning and therefore cannot be redefined.

27) What is closure subprogram?

Is a nested subprogram and its referencing environment, which together allow the subprogram to be called from anywhere in a program.

28) Define Sentences.

A string of a language.

29) Define Lexemes.

The lowest-level syntactic unit.

30) Define Token.

The smallest individual element in a program.

31) A language can be defined in two distinct ways, what are they?

- Recognition
- Generation

32) What is Language Generator?

A generator is a device used to generate the sentences of a language.

33) Define Grammar.

Is a section that discusses the formal language generation mechanism.

34) Define Parse tree.

Represents the syntactic structure of a program.

35) Define Ambiguity.

A grammar that generates a sentential form for which there are two or more distinct parse trees.

36) There are three approaches to build a lexical analyzer, what are they.

- Write a formal description of the token patterns of the language using a descriptive language related to regular expressions.
- Design a state transition diagram that describes the token patterns of the language and write a program that implements the diagram.
- Design a state transition diagram that describes the token patterns of the language and hard-construct a table-driven implementation of the state diagram.

37) Define Regular grammar.

Finite automata can be designed to recognize members of a class of language.

38) Define Finite automata.

State diagram of the form used for lexical analyzers are representations of a class of mathematical machines

39) Define Parsing.

Is the part of the process of analyzing syntax that is referred to as syntax analysis.

40) What are the two broad classes.

- **Top-down:** Parse tree is built from roots to leaves.
- **Bottom-up:** Parse tree is built from leaves to roots.

41) Define Special word.

Special word is used to separate the syntactic part of statements and program.

42) Define Aliases.

When more than one variable name can be used to access the same memory location.

43) What is Binding and Binding Time.

- **Binding:** Is an association between an attribute and an entity
(Such as between a variable and its type).
- **Binding Time:** Is the moment in the program's life cycle when this association occurs.

44) Variable declarations:

- **Explicit declaration:** Is a program statement used for declaring the types of variables.
- **Implicit declaration:** Is a default mechanism for specifying types of variables (the first appearance of the variable in the program).

45) Storage binding:

- **Allocation:** The process of binding a variable to an available memory cell.
- **Deallocation:** The process of unbinding a variable from a memory cell.

46) Type inference?

Is another kind of implicate declarations that uses context.

47) Elaboration?

Creates, initializes and runs a task.

48) Define Static scoping.

The scope of the variable can be determined statically (before execution).

49) Define Dynamic scoping.

The scope of the variable can be determined dynamically (during run-time).

50) block-structured language?

Permits an array with adjustable length.

51) What is named constant?

Is a variable that is bounded to a value only once.

52) Define Initialization.

Is the binding of a variable to a value at the time it is bounded to a storage.

53) What is Von Neumann Architecture?

Is a prevalent computer architecture used to design a language.

54) Language Categories:

- ❖ **Imperative:** Is a software development model where functions are coded in every step to solve problems.
- ❖ **Functional:** Main mean of making computations by applying functions to given parameters
- ❖ **Object-Oriented:** A computer programming model that organizes software designs around data.
- ❖ **Markup:** Used to specify the layout of information in web documents.

55) Implementation Methods:

- ❖ **Compiler:** Programs are translated into machine language.
- ❖ **Pure interpretation:** The program is translated into machine code for a particular CPU one instruction at a time and executes immediately
- ❖ **Hybrid Implementation System:** A compromise between compiler and pure interpretation.

56) Compilation steps:

- I. **Lexical Analysis:** Converts characters in the program into lexical units.
 - Lexical units of a program are identifiers, special words and punctuations symbols.
- II. **Syntax Analysis:** Converts characters in the program into parse tree.
 - There parse trees represent the syntactic structure of the program.
- III. **Semantic Analysis:** Generates intermediate code.
 - Intermediate languages sometimes look very similar to **assembly language** and in fact sometimes are actually **assembly language**.
- IV. **Code Generation:** Generates machine code.

57) Fetch-Execute cycle

initialize the program counter

repeat

fetch the instruction pointed by the counter

increment the counter

decode the instruction

execute the instruction

end repeat

58) Program counter?

A register that maintains the address of the next instruction to be executed.

59) What are Programming Macros?

Are commands used to specify that code from another file is being included.

#include #define

60) What are programming environments?

The collection of tools used in software development.

61) Numeric types:

- **Integer:** whole numbers including 0 and negatives.
- **Floating-point:** A decimal number that represents a signed real number.
- **Complex:** An expansion of complex macros in <complex.h>
- **Decimal:** Fixed-point decimal data type.
- **Boolean:** Is a data type that can be either true or false.
- **Character:** Is a data type that stores a character.

62) String length options:

- **Static length string:** The length is set when the string is created.
- **Dynamic length string:** The length of the string can vary with no maximum.
- **Limited Dynamic length string:** The length of the string can vary, however there is a declared fixed maximum.

63) Enumeration type?

All possible values, which are named constants, are provided in the definition.

64) What is an Array?

A series of memory locations with each of which holding a single item of data.

65) Define subscript.

The elements in an array.

66) Array categories:

- **Static array:** Is one in which the subscript ranges are statically bound and storage allocation is static (done before run-time).
- **Fixed stack-dynamic array:** Is one in which the subscript ranges are statically bound but allocation is done during execution.
- **Stack-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic (during execution).
- **Fixed heap-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic, but they are both fixed after storage is allocated.
- **Heap-dynamic array:** Is one in which the subscript ranges are dynamically bound, and the storage allocation is dynamic, and can change any number of times during the array's lifetime.

67) Associative Array?

Is an unordered collection of data elements that are indexed by an equal number of values called **key**.

68) Record?

Is a collection of data elements in which the individual elements are identified by names.

69) Union?

Is a type whose variables are allowed to store different type values at different times during execution.

70) Pointer?

Is a variable that stores the memory address of another variable as its value.

71) Selection statement?

Provides the means of choosing between two or more paths of execution.

- **Two-way selector:** An example of that is an “if statement” if the condition is satisfied a certain path is taken otherwise it is forced to take the other path.
- **Multiple-way selector:** Allows selection of one or more statements or group of statements, an example of that is “switch statement”.

72) Iterative statements

The repeated execution of the statement or a group of statements.

- **Counter-controlled loop:** Has a loop variable and a way of specifying the number of repetitions of the loop.
- **Logical-controlled loop:** Repetition control is based on Boolean.
- **User-located loop control mechanism:** Programmer decides when the loop ends and continues.
- **Statement-controlled loop:** Used to perform looping operations until the given condition is true.

73) Unconditional branch statement?

Transfer execution to a specified location in the program.

74) What is a Subprogram?

Subprograms are the fundamental building blocks of programs.

75) What is a Subprogram call?

The explicit request that a specific subprogram be executed.

76) What is a Subprogram header?

Is the first part of the subprogram and serves several purposes such as identifying the name of the subprogram and declare the local variables.

77) Local and global variables?

- **Local variable:** The variables that are defined inside a subprogram.
- **Global variable:** The variables that are defined outside that given function.

78) Subprogram types:

- **Function:** A block of code that only runs when it is called.
- **Procedure:** A collection of statements that define parameterized computations. (Has not return value).

79) Parameter passing methods:

- **Pass-by-value: (In-Mode)**
 - ❖ The value of the actual parameter is used to initialize the corresponding formal parameter.
- **Pass-by-result: (Out-Mode)**
 - ❖ Just before control is transferred back to the caller, the value of the formal parameter is transmitted back to the actual parameter.
- **Pass-by-value-result: (In-Out-Mode)**
 - ❖ Is a combination of Pass-by-value and Pass-by-result.
- **Pass-by-reference: (In-Out-Mode)**
 - ❖ Passing the memory address of where a variable is stored.
- **Pass-by-name: (In-Out-Mode)**
 - ❖ Is one where the variable name is passed.

80) Types of binding:

- **Static binding:** Is the binding that can be resolved at compile time.
- **Dynamic binding:** Is the binding that can be resolved at run time.

81) What is a nested subprogram?

Function which is defined within another function.